General rules of Python syntax

The commands that make up a program must be written according to certain rules. The computer must recognize commands unambiguously. It must understand where one command ends and where the next command begins. For this purpose, the following rules must be observed.

The rule	***Correct***	***Incorrect***

Rule of order
Commands will be executed in turn if they are written exactly under each other.
	`a = "Hello, "`
	`b = "world!"`
	`print(a+b)`
		` a = "Hello, "`
		`b = "world!"`
		` print(a+b)`

Rule of beginning
Each new command is written on a new line.
	`a = "Hello, "`
	`b = "world!"`
	`print(a)`
	`print(b)`
		`a = "Hello, "`
		`b = "world!"`
		`print(a)print(b)`

Neatness rule
Lowercase (small) letters cannot be replaced by uppercase (capital) letters.
	`a = "Hello, "`
	`b = "world!"`
	`print(a+b)`
		`a = "Hello, "`
		`b = "world!"`
		`PrInt(A+b)`

Print information to the screen

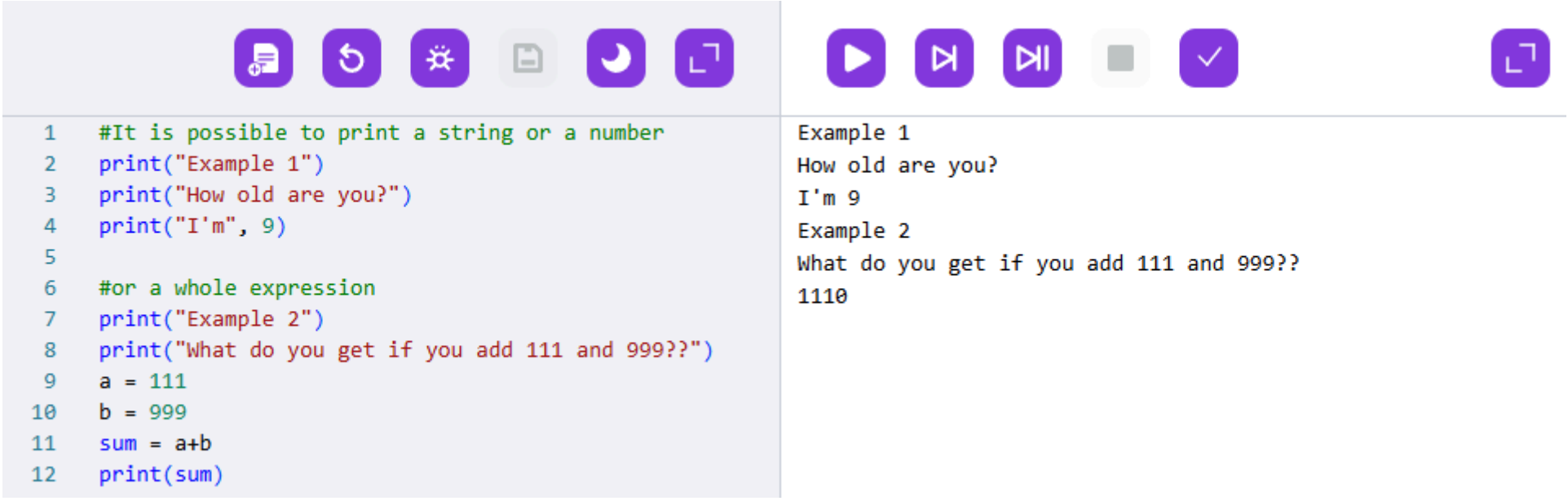
Command:

```
print()
```

Description:

The print() function is needed to print to the screen what is inside the brackets. This can be strings, numbers, variables, etc. To print a **string**, you need to write it in quotes - " " or ' ' (equal on both sides) and insert it into print(). To print a **number** or a **variable**, just paste it into print(). If you need to print multiple arguments (a string, a number and a variable), you just need to separate them inside the brackets with a comma.

How to use:



```
#It is possible to print a string or a number
print("Example 1")
print("How old are you?")
print("I'm", 9)

#or a whole expression
print("Example 2")
print("What do you get if you add 111 and 999??")
a = 111
b = 999
sum = a+b
print(sum)
```

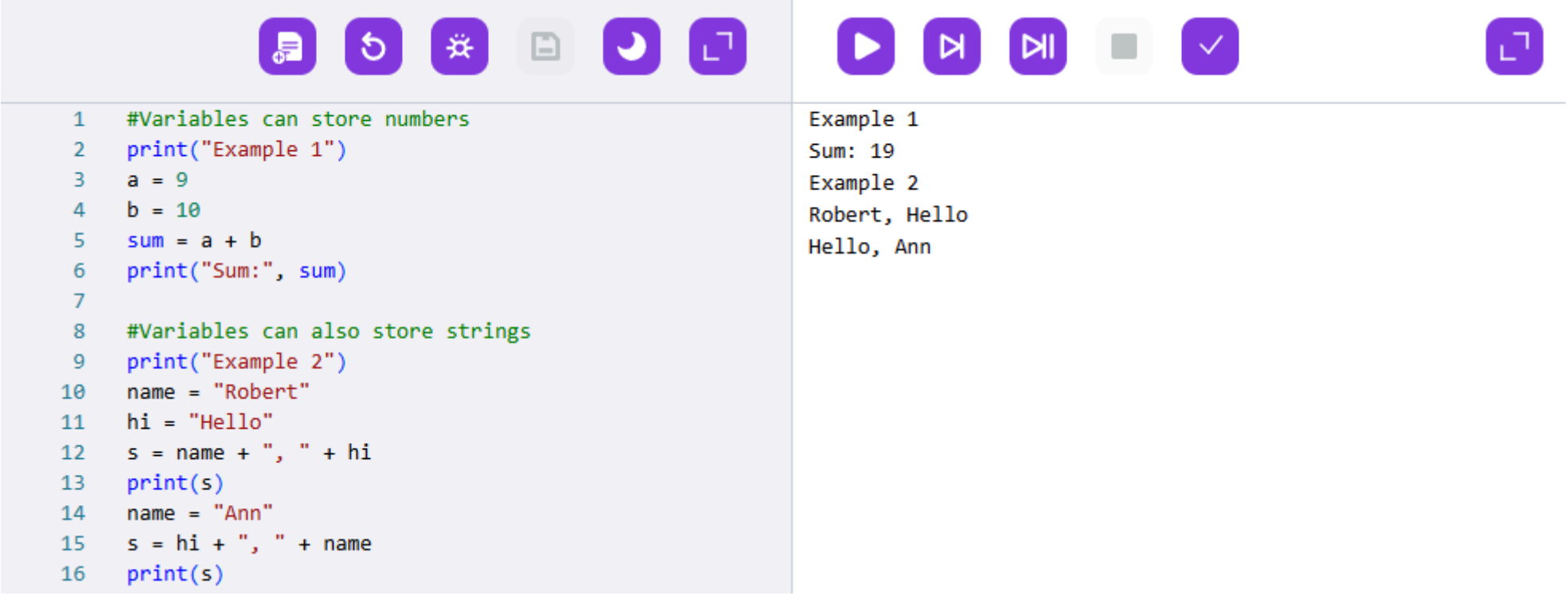
```
Example 1
How old are you?
I'm 9
Example 2
What do you get if you add 111 and 999??
1110
```

Variables

Description:

A variable is a data element that has a name. A variable is needed to store data that can be changed in a program. To use a variable, you must give it a name and an initial value. The assignment operator "=" sets the initial value of the variable. Variables can store numbers, strings, etc. Only Latin (English) characters can be used in the variable name. Only after a variable has been created and assigned an initial value can it be manipulated.

How to use:



```
#Variables can store numbers
print("Example 1")
a = 9
b = 10
sum = a + b
print("Sum:", sum)

#Variables can also store strings
print("Example 2")
name = "Robert"
hi = "Hello"
s = name + ", " + hi
print(s)
name = "Ann"
s = hi + ", " + name
print(s)
```

```
Example 1
Sum: 19
Example 2
Robert, Hello
Hello, Ann
```

Entering data into a program

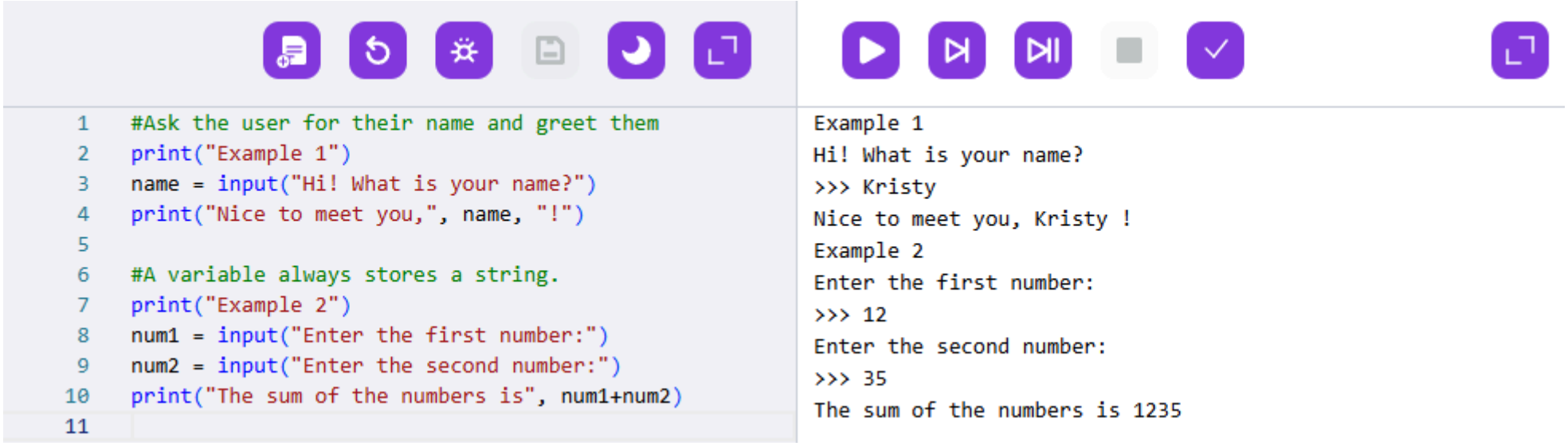
Command:

```
input()
```

Description:

The input() function is needed to pass data from the user to the program. Inside the brackets, a message is given to the user, prompting them to enter some information. The entered data is written to a variable and the next command is executed. This function always writes a **string** to the variable.

How to use:



```
#Ask the user for their name and greet them
print("Example 1")
name = input("Hi! What is your name?")
print("Nice to meet you,", name, "!")

#A variable always stores a string.
print("Example 2")
num1 = input("Enter the first number:")
num2 = input("Enter the second number:")
print("The sum of the numbers is", num1+num2)

```

```
Example 1
Hi! What is your name?
>>> Kristy
Nice to meet you, Kristy !
Example 2
Enter the first number:
>>> 12
Enter the second number:
>>> 35
The sum of the numbers is 1235
```

Data types

Description:

You can execute different operations on different types of data. For example, all arithmetic operations can be executed on numbers, strings can be printed or concatenated. Therefore, in order for the computer to unambiguously understand the programmer, a different name was invented for each data type. You can determine what type a variable belongs to by using the type() function. Below are some of the data types.

Data type	**What you can do**	**What you can't do**
Numbers: ● **int** - an integer, ● **float** is a fractional number.	Execute all arithmetic and logical operations: ● addition (+), ● multiplication (*), ● division (/), ● integer division (//), ● remainder of division (%), ● raising to a degree (**), ● comparison .	Attempt to perform arithmetic operations on other data types: ● adding a number and a string, ● raise a string to a power, ● similar to other arithmetic operations.
Strings: ❏ **str** is a string.	❏ print to the screen, ❏ glue (+), ❏ duplicate (*), ❏ compare.	❏ execute arithmetic operations on two strings.
Logical variables: ❖ **bool**.	❖ compare.	

```python
a = "Hello!" #str type
b = 123 #int type
c = 2.5 #float type
d = True #bool type
name = input("What is your name?") #str type

print("Variable a:", a)
print("Variable b:", b)
print("Variable c:", c)
print("Variable d:", d)
print("Variable name:", name)
```

```
What is your name?
>>> Pete
Variable a: Hello!
Variable b: 123
Variable c: 2.5
Variable d: True
Variable name: Pete
```

Data type definition

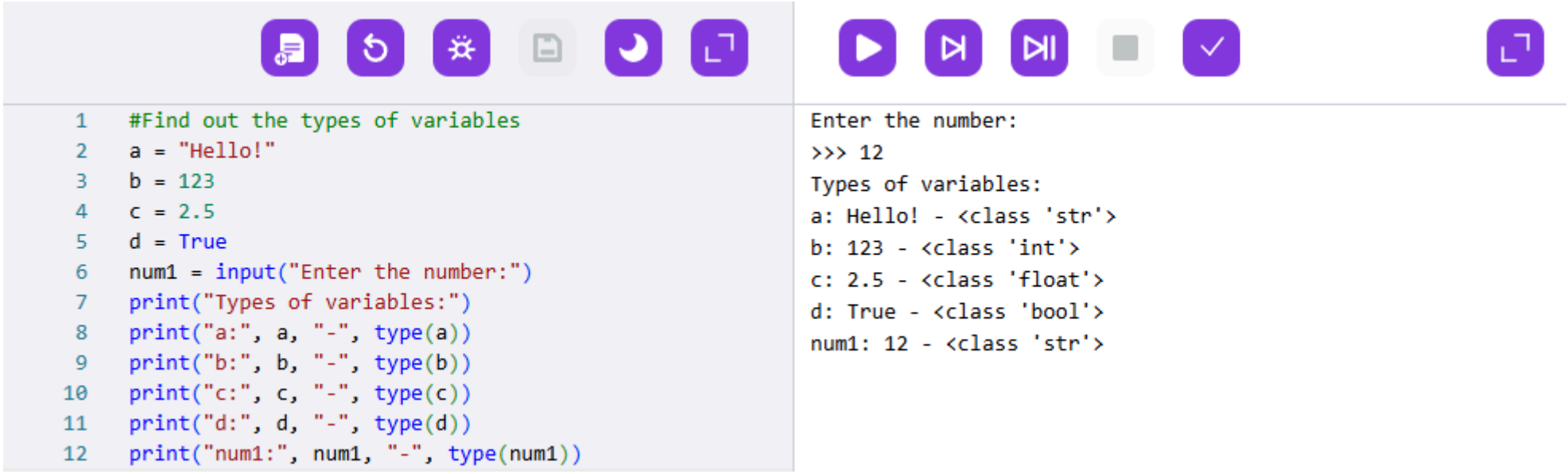
Command:

`type()`

Description:

The type() function is needed to determine the type of data. To find out what type a variable is, you need to specify it inside brackets.

How to use:



```
#Find out the types of variables
a = "Hello!"
b = 123
c = 2.5
d = True
num1 = input("Enter the number:")
print("Types of variables:")
print("a:", a, "-", type(a))
print("b:", b, "-", type(b))
print("c:", c, "-", type(c))
print("d:", d, "-", type(d))
print("num1:", num1, "-", type(num1))
```

```
Enter the number:
>>> 12
Types of variables:
a: Hello! - <class 'str'>
b: 123 - <class 'int'>
c: 2.5 - <class 'float'>
d: True - <class 'bool'>
num1: 12 - <class 'str'>
```

Conversion from string to number

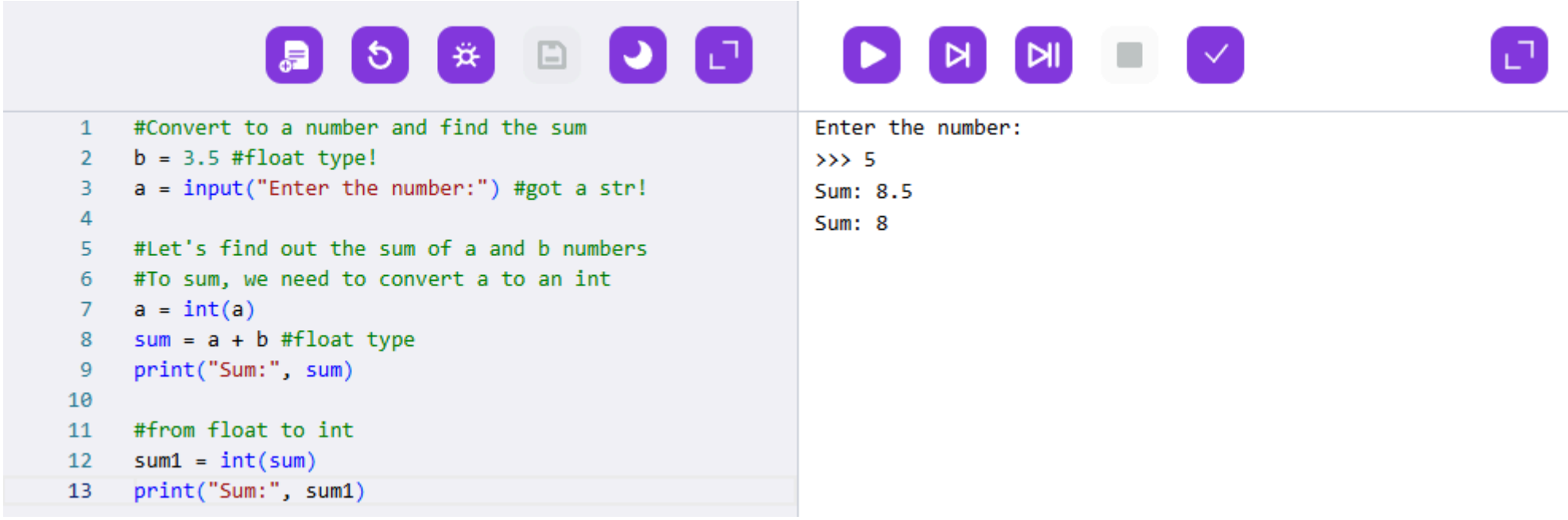
Command:

```
int()
```

Description:

The int() function converts a string into an integer. To convert a sequence of digits (str) into a number, you must specify it inside the brackets. Remember that this function cannot convert words into numbers.

How to use:



```
#Convert to a number and find the sum
b = 3.5 #float type!
a = input("Enter the number:") #got a str!

#Let's find out the sum of a and b numbers
#To sum, we need to convert a to an int
a = int(a)
sum = a + b #float type
print("Sum:", sum)

#from float to int
sum1 = int(sum)
print("Sum:", sum1)
```

```
Enter the number:
>>> 5
Sum: 8.5
Sum: 8
```

Conversion from number to string

Command:

```
str()
```

Description:

The str() function converts any data type into a string. To do this - you need to specify inside the brackets the data to be converted to a string.

How to use:

```
1  #Find the perimeter of a rectangle
2  lngth = int(input("Enter the lenght in cm"))
3  wdth = int(input("Enter the width in cm"))
4  P = 2*(lngth + wdth)
5  P = str(P)
6  message = "Rectangle perimeter " + P + " cm"
7  print(message)
```

```
Enter the lenght in cm
>>> 15
Enter the width in cm
>>> 12
Rectangle perimeter 54 cm
```

Hot keys

Description:

Programmers enter the texts of their programs using the keyboard. There are special keyboard combinations that make working on text much faster than using the mouse. These combinations are called **hotkeys**. To use them, you need to press a character key once while holding down the modifier key (**Ctrl** or **Shift**).

The table shows the most frequently used keyboard shortcuts.

Note:

MacOS uses the Command key instead of Ctrl.

Key combination	**Result (entertaining video about hotkeys)**
Tab	Replaces 4 spaces.
Ctrl + C	Copies to the clipboard.
Ctrl + V	Paste from clipboard.
Ctrl + Z	Cancel last action (not always possible).
Shift + move cursor	Select text.
Ctrl + X	Delete selected text with copying to clipboard.
Ctrl + <←> or <→>	Move one word in the text.
Ctrl + <Home>	Move to the beginning of the entire text.
Ctrl + <End>	Move to the end of the entire text.
Ctrl + or	Zooms in or out the screen.

Condition Check

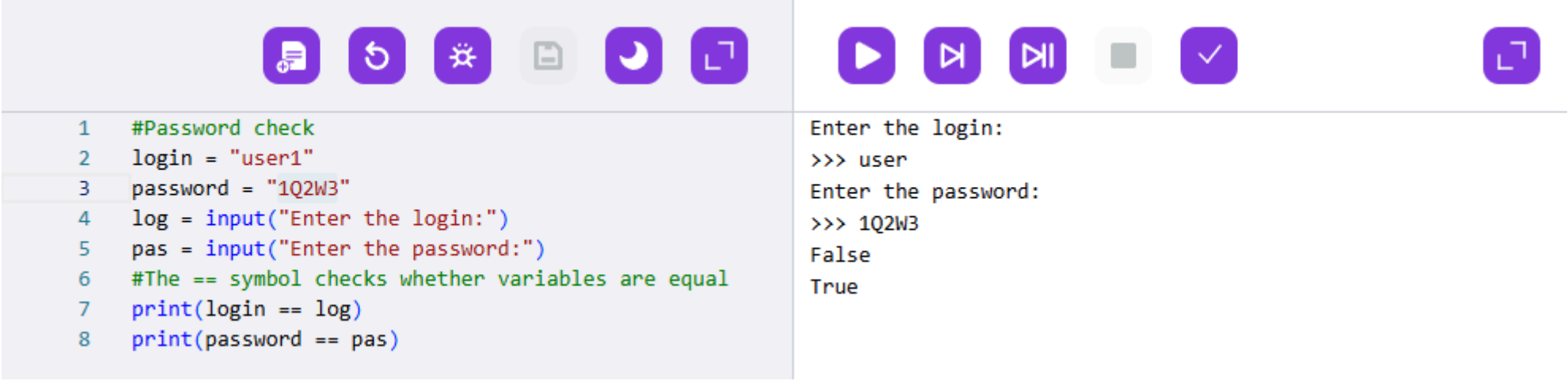
Description:

"Check Condition" is to understand whether a statement is true or false. Statements are expressions that can be true (True) or false (False). Comparison operators allow you to write statements.

Equal	Not equal	Less than	Greater than	Less than or equal	Greater than or equal to
==	!=	<	>	<=	>=

How to use:

For example, can compare the login and password that a user enters. If the login is correct, that is, all characters match, then the statement "login is correct" is true.



```
#Password check
login = "user1"
password = "1Q2W3"
log = input("Enter the login:")
pas = input("Enter the password:")
#The == symbol checks whether variables are equal
print(login == log)
print(password == pas)
```

```
Enter the login:
>>> user
Enter the password:
>>> 1Q2W3
False
True
```

Conditional if statement

Command:

```
if :
    command block 1
```

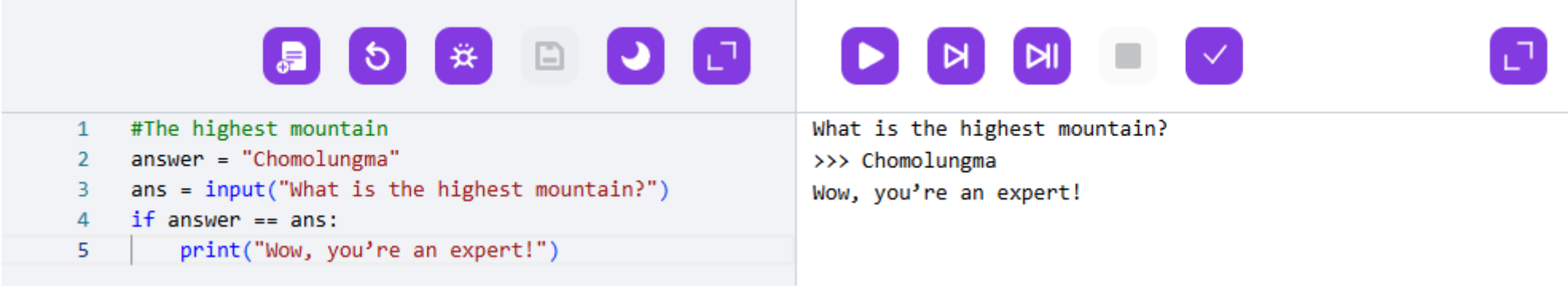
Description:

The **if** conditional statement is *a statement that* executes a specific set of commands if a condition is true. First, the value of the statement is calculated (True or False), then the commands to be executed next are selected. If the condition is true, "instruction block 1" is executed, otherwise (the condition is false) - the program continues.

Note:

- it is obligatory to put a "**:**" after the condition;
- all commands that come after the colon must be **shifted to the right by 4 spaces**.

How to use:



```
#The highest mountain
answer = "Chomolungma"
ans = input("What is the highest mountain?")
if answer == ans:
    print("Wow, you're an expert!")
```

```
What is the highest mountain?
>>> Chomolungma
Wow, you're an expert!
```

The if-else conditional statement

Command:

```
if :
    command block 1
else:
    command block 2
```

Description:

The **if-else** conditional statement *is an operator that execu*tes different sets of commands, depending on the truth of the condition. If the condition is true, "command block 1" is executed, otherwise (the condition is false) - "command block 2" is executed.

Note:

- It is obligatory to put **":"** after the condition and after else;
- all commands that come after the colon must be **shifted to the right by 4 spaces**.

How to use:

```
#The highest mountain
answer = "Chomolungma"
ans = input("What is the highest mountain?")
if answer == ans:
    print("Wow, you're an expert!")
else:
    print("Not quite!")
```

```
What is the highest mountain?
>>> Chogori
Not quite!
```

If-elif-else conditional statement

Command:

```
if <condition_1> :
    command block 1
elif
    command block 2
elif
    command block 3
else:
    command block 4
```

Description:

If there are multiple conditions, the conditional statement **if-elif-else** is used. If "condition_1" is false - "condition_2" is checked. If it is true - "command block 2" is executed. In this case, the program does not check the other conditions, but immediately moves on to the next command in the algorithm.

Note:

- it is obligatory to put the "**:**" symbol after else and all conditions;
- all commands that come after the colon must be **shifted to the right by 4 spaces**;
- **if** and **else** occur **only once** in the program, **elif** - as many times as necessary.

How to use:

```python
#Names of large numbers (greater than a million)
num = int(input("Enter the number of zeros"))
if num < 12:
    print("Billion")
elif num < 15:
    print("Trillion")
elif num < 18:
    print("Quadrillion")
elif num < 21:
    print("Quintillion")
elif num < 23:
    print("Sextillion")
else:
    print("What a large number!")
    print("It is larger than septillion!")
```

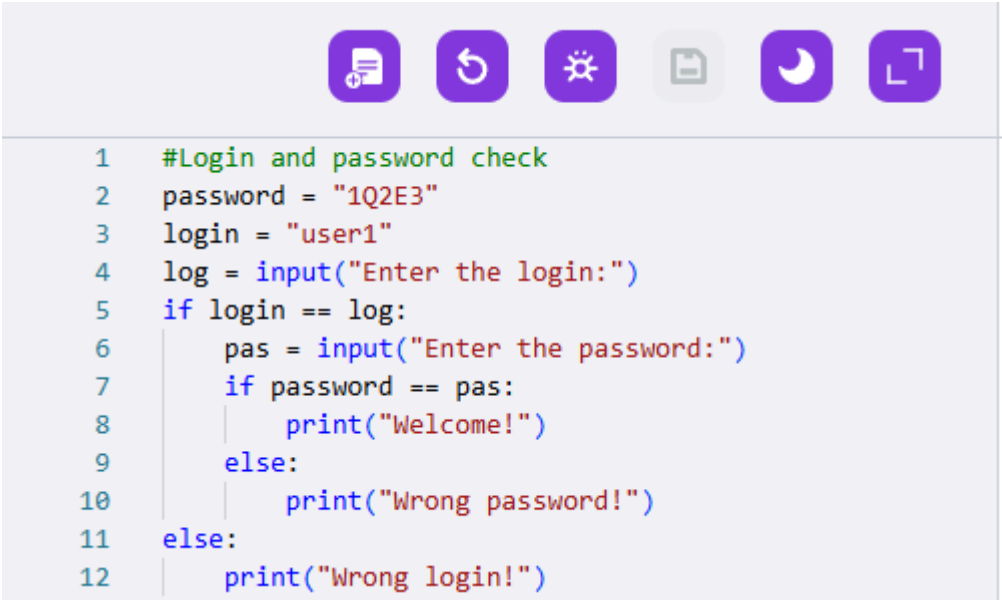
```
Enter the number of zeros
>>> 20
Quintillion
```

Nested conditional operator

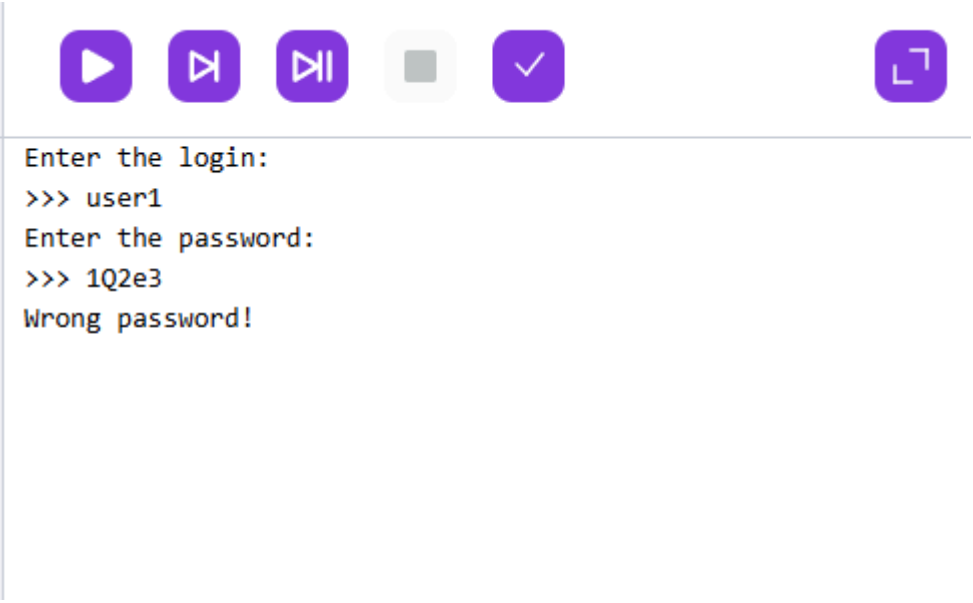
Description:

A nested conditional statement is used when you want to perform several different actions based on several conditions. This means that within one conditional statement there may be one or more other conditional statements that check other conditions.

How to use:



```
1   #Login and password check
2   password = "1Q2E3"
3   login = "user1"
4   log = input("Enter the login:")
5   if login == log:
6       pas = input("Enter the password:")
7       if password == pas:
8           print("Welcome!")
9       else:
10          print("Wrong password!")
11  else:
12      print("Wrong login!")
```



```
Enter the login:
>>> user1
Enter the password:
>>> 1Q2e3
Wrong password!
```

Loop with condition

Command:

```
while <condition>:
    command block 1
```

Description:

A loop is a multiple repetition of a specific set of commands.

An iteration is a single repetition of a set of loop commands.

A **while** loop is *a loop that executes as long as a* specified condition is true. The **while** keyword indicates that a loop with a condition follows. As soon as the condition becomes false - the loop will terminate and move on to the next instruction.

How to use:

```
#Keep asking for the password until the correct one is
password = "1Q2E3"
pas = " "
while password != pas:
    pas = input("Enter the password:")
print("Welcome!")
```

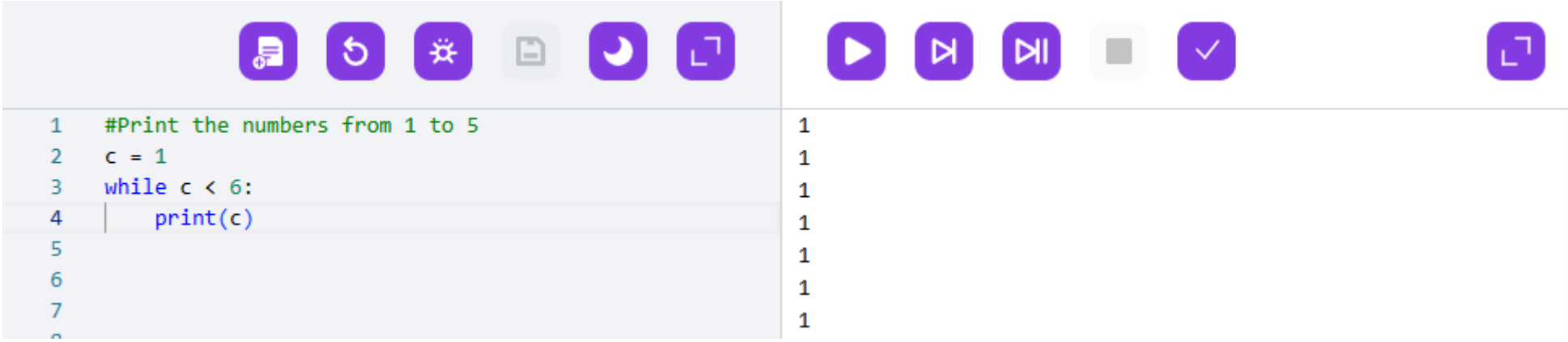
```
Enter the password:
>>> 1QIon
Enter the password:
>>> MjR5W
Enter the password:
>>> 1Q2E3
Welcome!
```

Infinite loop*.

Description:

The **while** loop becomes **infinite** if the loop condition never becomes false. Basically, it becomes infinite if a novice programmer forgets to put a counter. But an infinite loop can be used in a "Clock" program if you need to constantly update the time.

How to use it:



Counter

Description:

If you need to perform a certain number of repetitions or count how many attempts it took the user, you need to use a counter. To do this, you need to create a variable that will store information about repetitions. This variable is specified before the loop, and then it is changed inside the loop body for each repetition.

How to use:

```
#Count the number of attempts
password = "1Q2E3"
pas = " "
c = 0
while pas != password:
    pas = input("Enter the password:")
    c = c + 1
print("Number of attempts:", c)
```

```
Enter the password:
>>> W3f5h
Enter the password:
>>> a3Cf
Enter the password:
>>> 1Q2E3
Number of attempts: 3
```

```
#Print the numbers from 1 to 5
c = 5
while c > 0:
    print(c)
    c = c - 1
```

```
5
4
3
2
1
```

Functions

Command:

```
def function_name(argument_1, argument_2, ...):
    command block
```

Description:

A function is an algorithm composed of already known commands and given a name that reflects its operation.

A function must be defined: describe the data it receives when it starts working and the commands it executes. To define a function, the code uses the reserved word **def**, which is an abbreviation of the English word define.

To use a function, you must specify its name, put brackets and specify its arguments (if any). When the program reaches the mention of the function name, the function code will be executed and when it is finished, the program will return to the next command.

How to use:

```
#Find the perimeter of a rectangle
#Function creation
def perimeter():
    a = 15 #width
    b = 12 #length
    p = 2 * (a + b) #perimeter
    print("Rectangle perimeter", p)

print("Find the perimeter of a rectangle")
#Function usage
perimeter()
print("Well done!")
```

```
Find the perimeter of a rectangle
Rectangle perimeter 54
Well done!
```

Function arguments

Command:

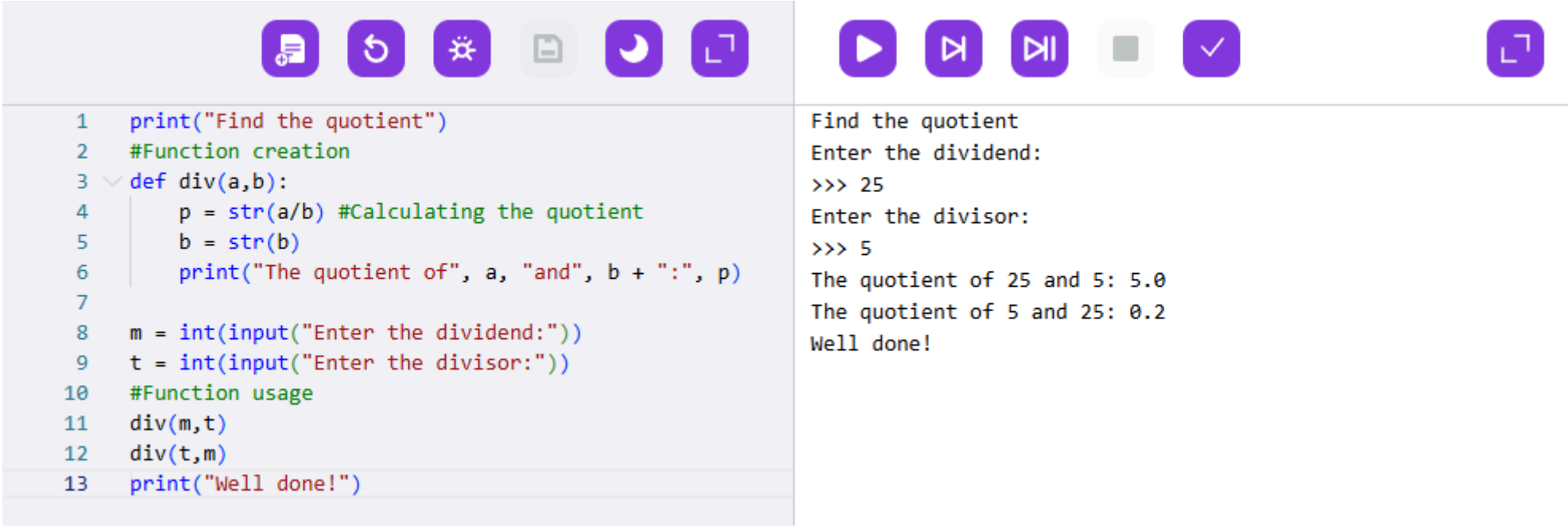
```
def function_name(argument_1, argument_2, ...):
    block of commands
```

Description:

The arguments of a function are data that is passed from the program and used

They are used inside the function. For example, the print(argument_1, argument_2, ...) function receives the data to be printed. The input parameters (arguments) must be specified in the correct order, otherwise the algorithm will not execute correctly.

How to use:



```
print("Find the quotient")
#Function creation
def div(a,b):
    p = str(a/b) #Calculating the quotient
    b = str(b)
    print("The quotient of", a, "and", b + ":", p)

m = int(input("Enter the dividend:"))
t = int(input("Enter the divisor:"))
#Function usage
div(m,t)
div(t,m)
print("Well done!")
```

```
Find the quotient
Enter the dividend:
>>> 25
Enter the divisor:
>>> 5
The quotient of 25 and 5: 5.0
The quotient of 5 and 25: 0.2
Well done!
```

Functions that return values

Command:

`return <value>`

Description:

return is a command that allows you to pass a value from a function to a program. The value returned by the function can be used further in the program, while all temporary data from the function will be removed.

To pass values from a function, you must specify after the return command what you want to return to the program.

How to use:

```
1   #The sum of the largest numbers
2   #Function creation
3   def num():
4       a = int(input("Enter the first number"))
5       b = int(input("Enter the second number"))
6       if b < a:
7           return a
8       else:
9           return b
10  #Assigning values returned by a function to variables
11  p = num()
12  v = num()
13  sum1 = p + v
14  print("The sum of the largest numbers:", sum1)
```

```
Enter the first number
>>> 14
Enter the second number
>>> 15
Enter the first number
>>> 12
Enter the second number
>>> 17
The sum of the largest numbers: 32
```

Module random. The randint function

Command:

```
from random import *
from random import randint
```

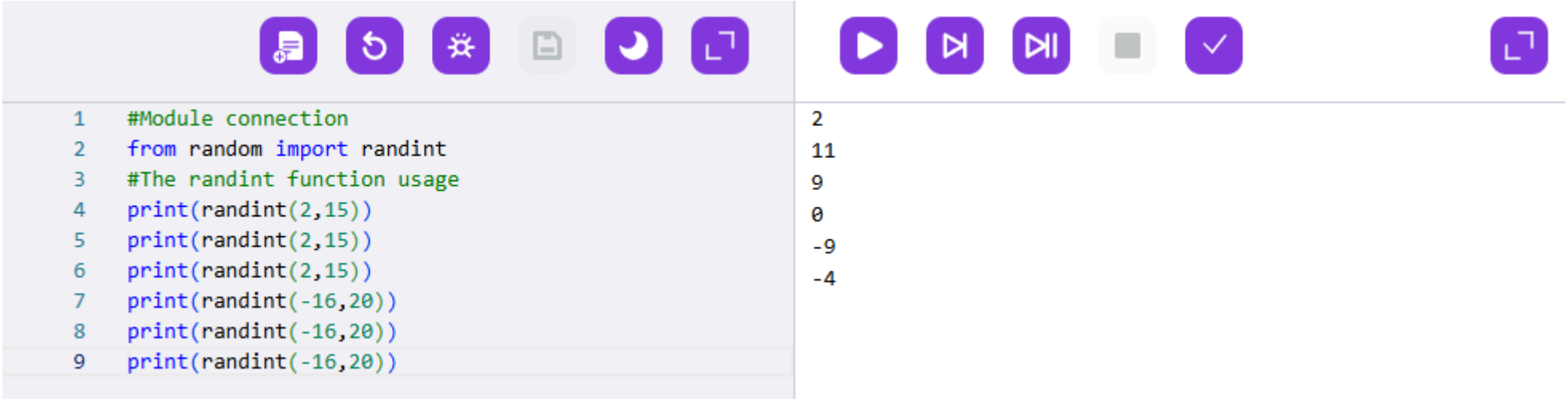
Description:

A module is a program file that you can plug into another program and use the functions written in that module.

For example, **random** is a module that provides functions for generating random numbers, symbols, etc. The function **randint(a,b)** is defined in this module. It creates and returns a random integer from a to b.

You can connect only one function from the module (for example, import **randint**) or all functions at once (import ***).

How to use:



```
#Module connection
from random import randint
#The randint function usage
print(randint(2,15))
print(randint(2,15))
print(randint(2,15))
print(randint(-16,20))
print(randint(-16,20))
print(randint(-16,20))
```

```
2
11
9
0
-9
-4
```

Connecting a module

Command:

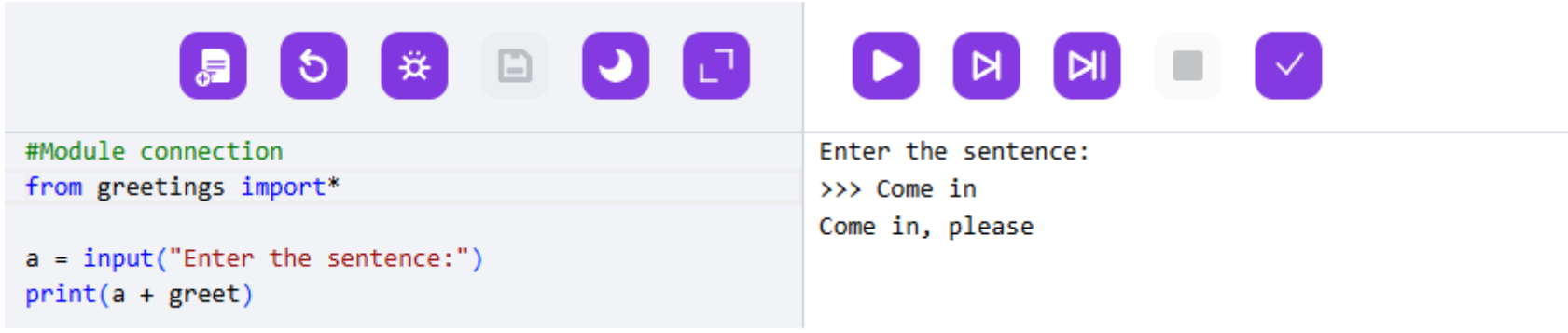
```
from module_name import
from module_name import variable_name#load a variable from the module
from module_name import function_name     #load function from the module
from module_name import *                 #load all functions from the module
import module_name                        #load all functions from the module
```

Description:

A module can be connected to a program in many different ways. You can use one of the constructions described above.

For example, if import starts with **from**, then after import you should specify function_name or *. If the import starts with **import**, you should specify the function_name after it. If there are several functions in the module and you need to load specific ones, you must specify the name of these functions after import with a comma.

How to use:



```
#Module connection
from greetings import*

a = input("Enter the sentence:")
print(a + greet)
```

```
Enter the sentence:
>>> Come in
Come in, please
```

Creating and using a module on the platform

Description:

A module is a program file that can be plugged into another program. Modules are needed to store frequently used functions.

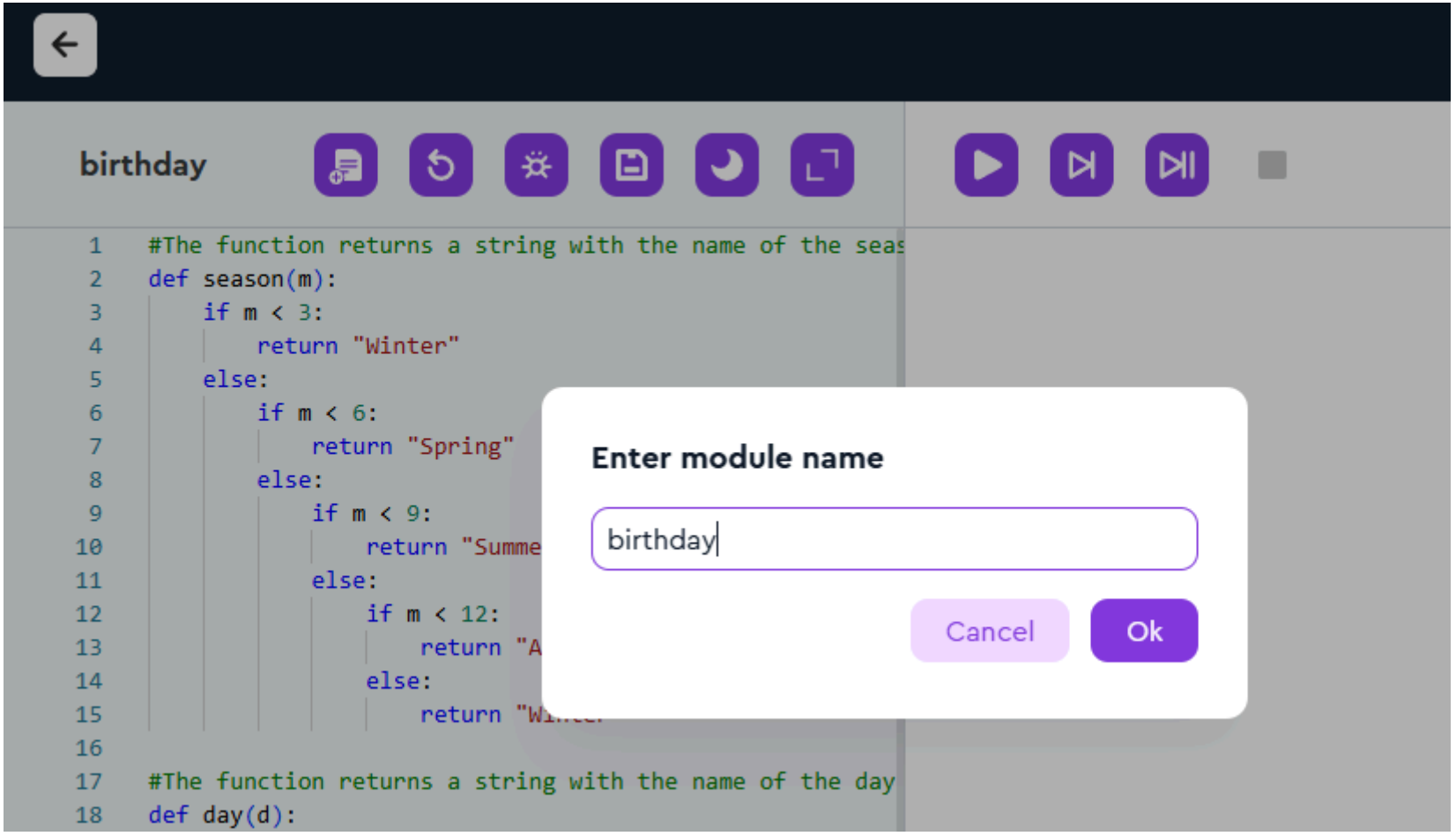
The program is needed to run directly, and the module is needed to load (connect) it into the program.

In order to create a module, you need to save the program to a file.

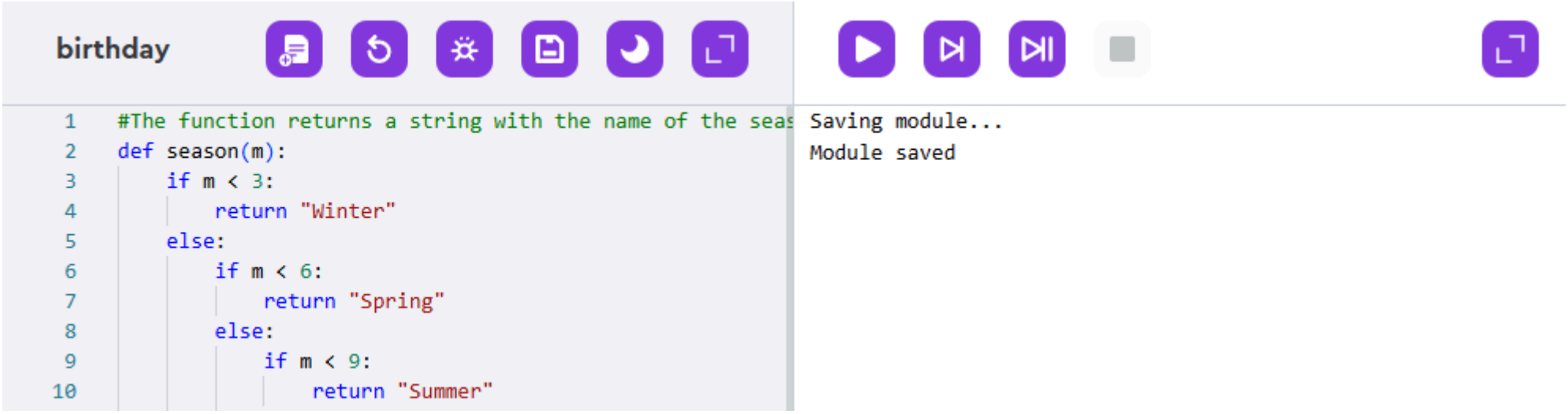
How to create, save and connect a module on the platform:

1. Click on the Save icon.

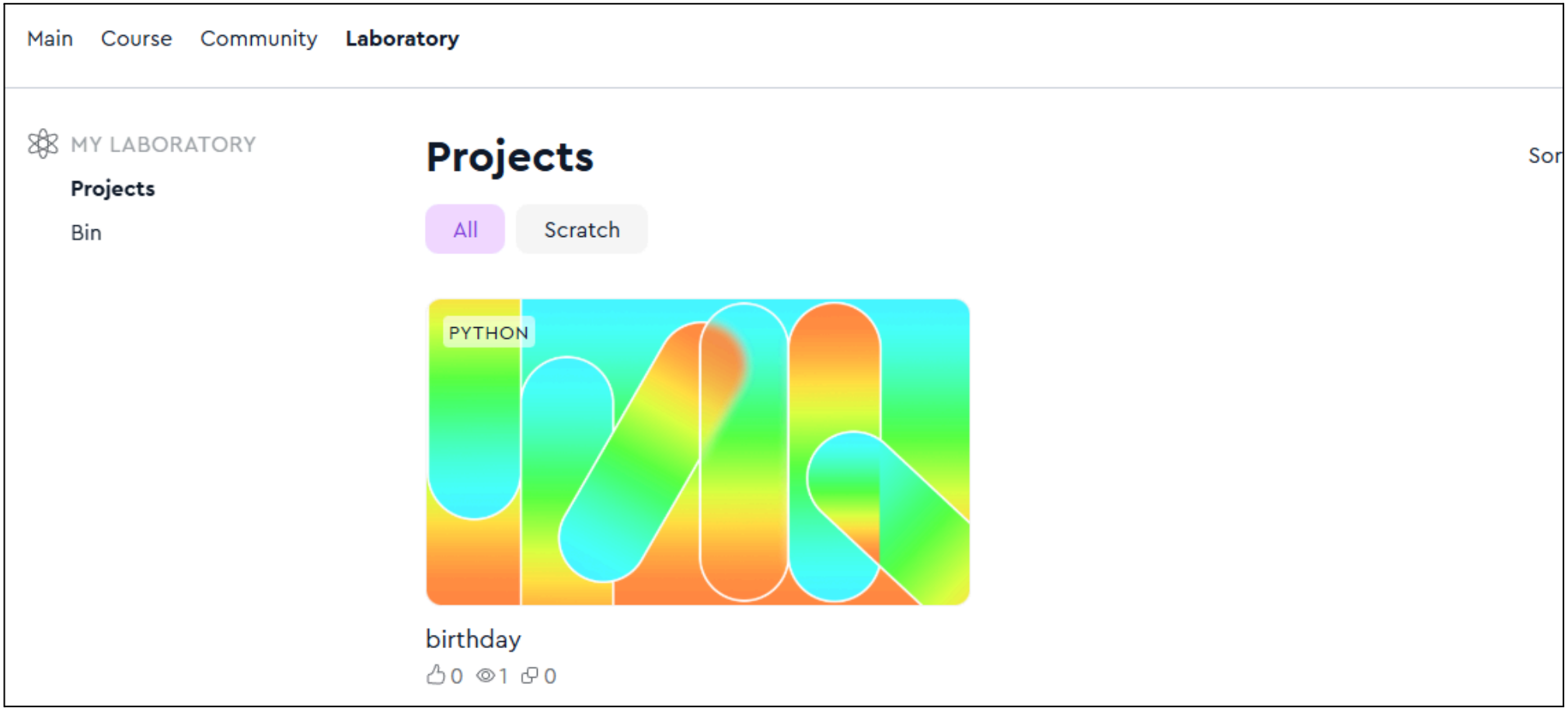
2. In the menu that appears, enter the name of the module (the name should reflect the meaning of what the module does) and click "OK".



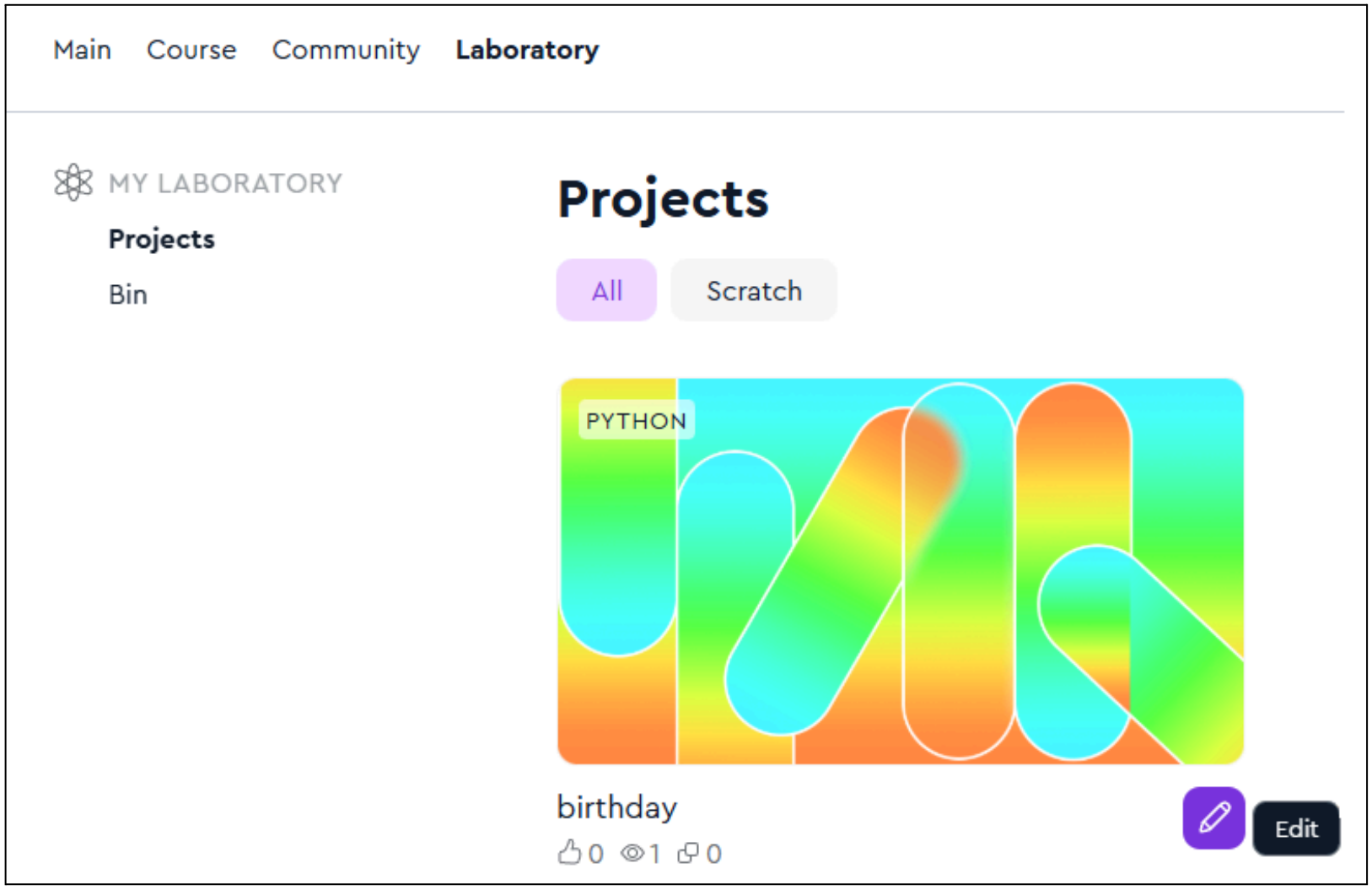
3. In the output field, you will see the label "Saving module... Module saved", which means that the module has been saved.



4. After saving, the module will be stored in the Laboratory.



5. If you need to make changes to the module, you need to go to the Laboratory and open the required file. To do this, click on the brush sign ("Edit").



6. Now the saved module can be used in the program. To do this, you just need to connect it (specify its name in the construction `from module_name import *`).

Task

```
1   from random import randint
2   from birthday import*
3
4   #The program asks for a month number and prints its name
5   m = int(input("Enter the month number of your birthday:"))
6   s1 = "It's " + season(m) + "."
7   print(s1)
8   #The program asks for a day-of-the-week number and prints its name
9   d = int(input("Enter the number of the day of the week you were born on:"))
10  s2 = "It's " + day(d) + "."
11  print(s2)
```

```
Enter the month number of your birthday:
>>> 09
It's Autumn.
Enter the number of the day of the week you were born on:
>>> 4
It's Thursday.
```

Turtle module

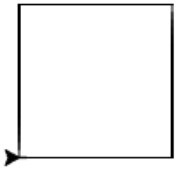
Description:

The turtle module is a module that allows you to create graphical objects (drawings) by an executor in a special window. **The executor** of the algorithm is a turtle. The image will consist of pixels - minimal indivisible elements (points). There are a number of commands that can be used to create various graphical objects.

To use this module, you need to connect it. Then you can use commands to draw graphical objects. The commands forward(), left(), rigth(), etc. will be discussed further. To fix the drawing in the window after the program is finished, you should use the exitonclick() function, otherwise the window will close immediately after the program execution and we will not have time to see the created drawing.

How to use:

```
1   #Module connection
2   from turtle import *
3   #Draw a square
4   forward(100)
5   left(90)
6   forward(100)
7   left(90)
8   forward(100)
9   left(90)
10  forward(100)
11  left(90)
12  #Fix the image
13  exitonclick()
```



Executor. Appearance

Command:

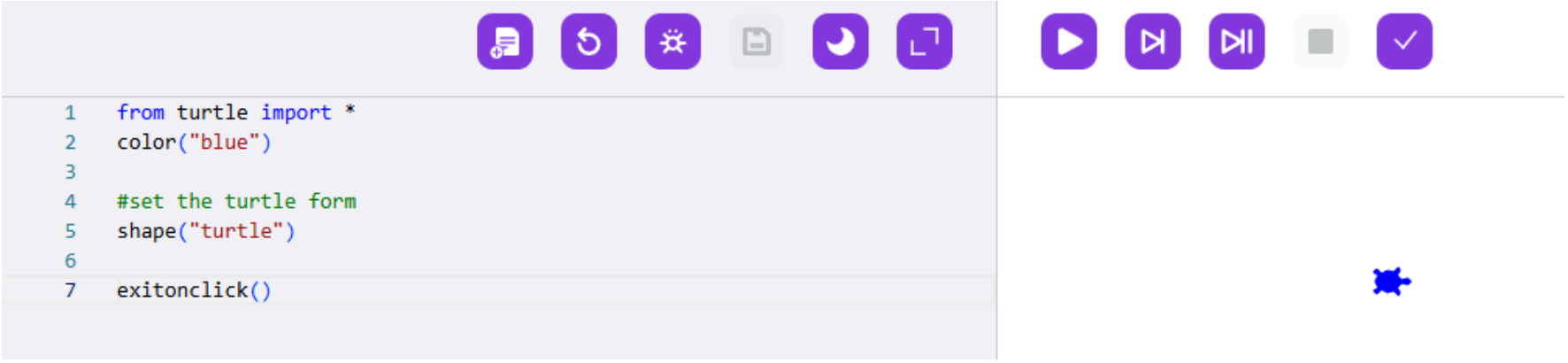
```
shape("<shape>")
```

Description:

The shape() command is used to change the shape of the executor. The appearance of the executor is a black arrow pointing in the direction of movement. At the start of the program, the executor always looks to the right. In the future - the executor will be called a turtle. There are several standard forms of the executor:

arrow	turtle	circle	square	triangle	classic
					

How to use:



```
from turtle import *
color("blue")

#set the turtle form
shape("turtle")

exitonclick()
```

Turtle movement

Command:

`forward(<number of pixels>)`

Description:

The forward() function moves the turtle forward, leaving behind a line. The length of this line is equal to the number of pixels specified in parentheses. By default, when moving, the turtle leaves a trace - a line. Also, the turtle starts its movement from the center of the screen, unless otherwise specified. When using the forward(100) command, a line with a length of 100 pixels will be drawn (see example).

How to use:

```
1  #Module connection
2  from turtle import *
3
4  #Move forward
5  forward(100)
6
7  #Fix the image
8  exitonclick()
```

The direction of the turtle's movement

Command:

```
left(<degrees>)
```

```
right(<degrees>)
```

Description:

If the turtle's direction of movement needs to be changed, the left() and right() functions are used. They rotate the turtle by the number of degrees specified in brackets, relative to the initial position. By default, the turtle looks to the right.

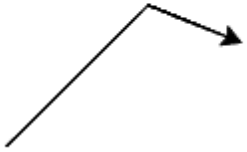
How to use:

```
#Module connection
from turtle import *

#Turn left 45 degrees
left(45)
forward(100)

#Turn right 67 degrees
right(67)
forward(50)

#Fix the image
exitonclick()
```



The color of the turtle

Command:

`color("<color>")`

Description:

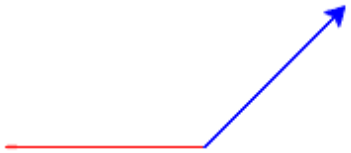
The color of the turtle can be changed using the color() function. This function gets one parameter, a string with the name of the color in English. After using this command, the executor will draw with the specified color until the color is changed to some other color. The default color of the turtle is black. The name of the colors can be seen below.

How to use:

```
#Module connection
from turtle import *

#Red color
color("red")
forward(100)
left(45)

#Blue color
color("blue")
forward(100)
exitonclick()
```



"red"	"green"	"blue"	"yellow"	"black"	"gray"	"violet"
red	green	blue	yellow	black	gray	purple


white gainsboro silver darkgray gray dimgray
whitesmoke lightgray lightcoral rosybrown indianred red maroon
snow mistyrose salmon orangered chocolate brown darkred
seashell peachpuff tomato darkorange peru firebrick olive
linen bisque darksalmon orange goldenrod sienna darkolivegreen
oldlace antiquewhite coral gold limegreen saddlebrown darkgreen
floralwhite navajowhite lightsalmon darkkhaki lime darkgoldenrod green
cornsilk blanchedalmond sandybrown yellow mediumseagreen olivedrab forestgreen
ivory papayawhip burlywood yellowgreen springgreen seagreen darkslategray
beige moccasin tan chartreuse mediumspringgreen lightseagreen teal
lightyellow wheat khaki lawngreen aqua darkturquoise darkcyan
lightgoldenrodyellow lemonchiffon greenyellow darkseagreen cyan deepskyblue midnightblue
honeydew palegoldenrod lightgreen mediumaquamarine cadetblue steelblue navy
mintcream palegreen skyblue turquoise dodgerblue blue darkblue
azure aquamarine lightskyblue mediumturquoise lightslategray blueviolet mediumblue
lightcyan paleturquoise lightsteelblue cornflowerblue slategray darkorchid darkslateblue
aliceblue powderblue thistle mediumslateblue royalblue fuchsia indigo
ghostwhite lightblue plum mediumpurple slateblue magenta darkviolet
lavender pink violet orchid mediumorchid mediumvioletred purple
lavenderblush lightpink hotpink palevioletred deeppink crimson darkmagenta

Line thickness

Command:

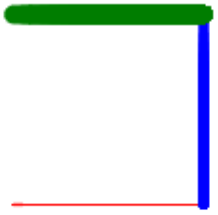
`width()`

Description:

To change the line thickness, you must specify the line thickness in pixels inside the width() function brackets. The default line thickness is 1 pixel.

How to use:

```
1   from turtle import *
2   color("red")
3   forward(100)
4   left(90)
5   #Width: 5 pixels
6   width(5)
7   color("blue")
8   forward(100)
9   left(90)
10  #Width: 10 pixels
11  width(10)
12  color("green")
13  forward(100)
14  exitonclick()
```



Raise and lower the pen

Command:

```
penup()
```

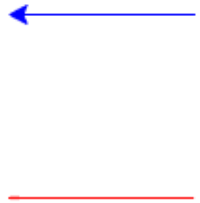
```
pendown()
```

Description:

Since the turtle leaves a trace when it moves, we can call it a feather (pen - "quill", "pen"). This pen can be raised and lowered if you need to move the executor to another part of the screen without a trace. The Turtle module has special functions that allow you to do this. To do this, use the penup() function to raise the pen, move the turtle to the desired part of the screen, and then lower it using the pendown() function.

How to use:

```
from turtle import *
width()
color("red")
forward(100)
left(90)
#Pen up
penup()
forward(100)
left(90)
#Pen down
pendown()
color("blue")
forward(100)
exitonclick()
```



Moving the executor around the screen

Command:

```
goto(X,Y).
```

Description:

The Turtle module has another function that allows you to move the executor around the screen. To do this, you need to specify the destination points inside the brackets in the format: goto(X coordinate, Y coordinate). When you run the program, the turtle starts moving from the center of the screen (0,0).

How to use:

Let the my_paint module have previously written a star() function that draws a star.

```
1   from turtle import *
2   from my_paint import star()
3
4   star() #the function draws a black star
5   penup()
6   #go to x: -100 y: -50 coordinates
7   goto(110, 75)
8   pendown()
9   color("red")
10  star() #red star
11  exitonclick()
```

Filling shapes with color

Command:

```
begin_fill()
end_fill()
```

Description:

The begin_fill() and end_fill() functions are needed to fill a shape. To color a shape, you should write the begin_fill() command before you start drawing it, and end_fill() at the end. It is important that the brackets of these functions are empty, they do not take any parameters as input.

How to use:

```
1  from turtle import *
2  color("blue")
3  begin_fill() #begin filling
4  forward(50)
5  right(144)
6  forward(50)
7  right(144)
8  forward(50)
9  right(144)
10 forward(50)
11 right(144)
12 forward(50)
13 right(144)
14 end_fill() #end filling
15 exitonclick()
```

What is OOP

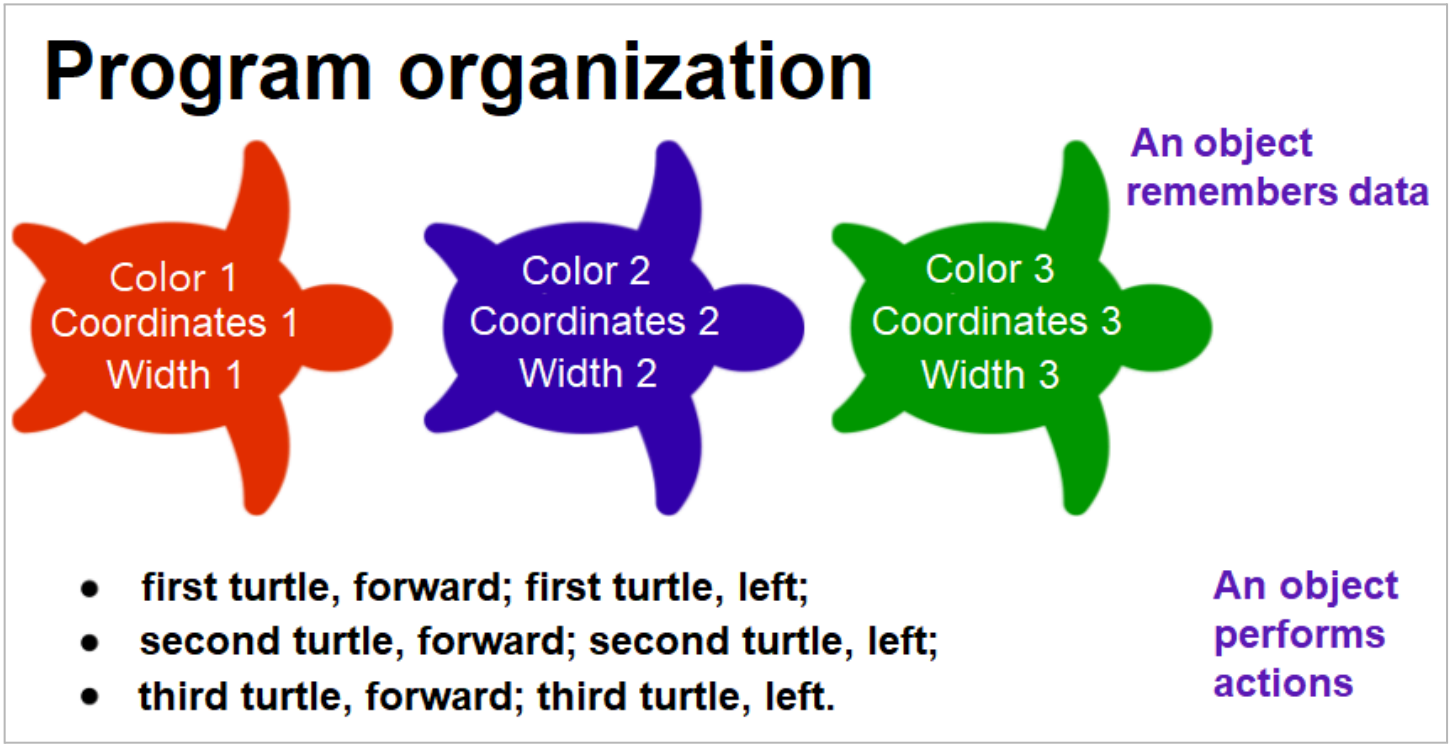
Description:

Object-oriented programming (OOP) is an approach to programming in which a program is viewed as a set of interacting objects. This approach is often used when implementing large projects, as each programmer can develop his own group of objects and only need to know how they will interact with others.

The basic concepts are "class", "object", "inheritance", etc. You need it to work with different types of objects. You have already worked with one type of object - a turtle. It has a set of properties (color, name) and skills to execute actions (moves, rotates, etc.).

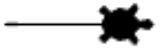
In OOP, you can create your own objects and methods, such as a game object.

Explanation diagram:



How to use:

```
1   from turtle import *
2
3   #Create turt object of Turtle class
4   #Object name turt
5   turt = Turtle()
6
7   #Set the property to the object - turtle form
8   turt.shape("turtle")
9
10  #Action of turt object - move forward
11  turt.forward(50)
12
13  exitonclick()
```



Creating an object

Command:

`<object name> = <object type>()`

Description:

An object is a set of data and actions that are treated as a single entity. To create an object you need to come up with a name and define the type to which it belongs. For example, you could create an object of type turtle. Objects should be named so that all developers understand what objects are responsible for.

Working with objects opens new opportunities in programming. For example, you can create several objects of the same type and work with them simultaneously or even create your own type of objects (we'll talk about this later).

How to use:

```
1   from turtle import *
2   #Creating an objects of Turtle class
3   turt1 = Turtle()
4   turt2 = Turtle()
5   #Actions of turt1 object
6   turt1.shape("turtle")
7   turt1.forward(50)
8   #Actions of turt2 object
9   turt2.color("red")
10  turt2.shape("circle")
11  turt2.back(50)
12
13  exitonclick()
```

Object methods

Command:

```
<object>.<method>()
```

Description:

A **method** is a function placed inside an object that is responsible for actions that the object can perform. For a turtle, this could be moving around the screen, changing shape, etc. To apply a method, it must be specified to the right of the object via a dot.

How to use:

```
1   from turtle import *
2   #Creating an objects of Turtle class
3   turt1 = Turtle()
4   turt2 = Turtle()
5   turt2.color("red") #turt2 color is red
6   #turt1 turns right using the right() method
7   turt1.right(35)
8   #turt1 moves forward using the forward() method
9   turt1.forward(50)
10  #turt2 moves to (50, 50) using the goto() method
11  turt2.goto(50, 50)
12  #turt2 moves back using the back() method
13  turt2.back(50)
```

Object properties

Command:

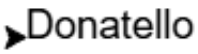
`<object>.<property>`

Description:

A property is a variable placed inside an object. Properties store necessary data about the object.

How to use:

```
1  from turtle import *
2  turt1 = Turtle()
3  #properties of turt1 object - name
4  turt1.name = "Donatello"
5  #using property - print name
6  turt1.write(turt1.name, font=('Arial', 14, 'normal'))
7  exitonclick()
```



The methods of the turtle object

Commands:

```
t = Turtle() #create object t
```

Command	Description
`t.forward()`	Moves the object forward a certain number of pixels.
`t.back()`	Move the object backwards by a certain number of pixels.
`t.left(),t.right()`	Rotates the object by a certain number of degrees relative to the original position.
`t.speed()`	Set the object's speed.
`t.write()`	Display the message on the screen.
`t.shape()`	Change the shape of the object.
`t.color()`	Change the color of the object.
`t.goto()`	Moving the object to specific window coordinates.
`t.penup(), t.pendown()`	Raising and lowering the pen.
`t.begin_fill(), t.end_fill()`	Beginning and ending of the fill.
`t.xcor(), t.ycor()`	Define the x and y coordinates of the object.
`t.clear()`	Clear the window.

Event handling

Description:

An event is information about what is happening in the "outside world". Information about an event is prepared by the program execution system. The external world can be understood as anything that can send some information. It can be a keyboard, a mouse, or another program.

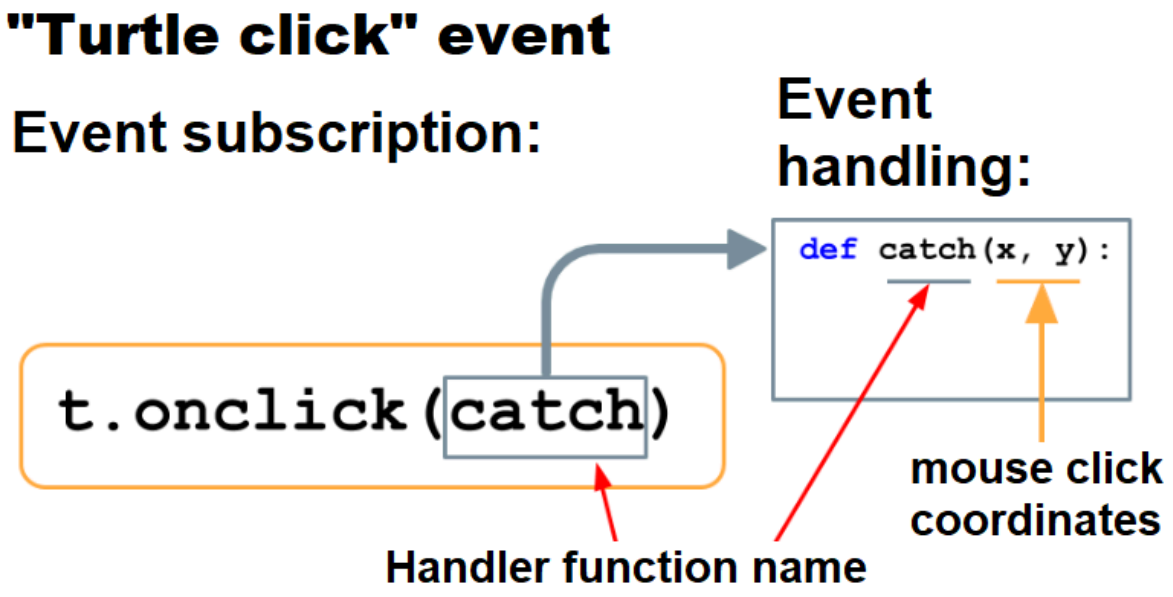
An event subscription is a program's request for an event important for it. As a rule, only necessary events are subscribed to (not all events of the "outside world" are important).

An event handler is an algorithm that describes the reaction to an event.

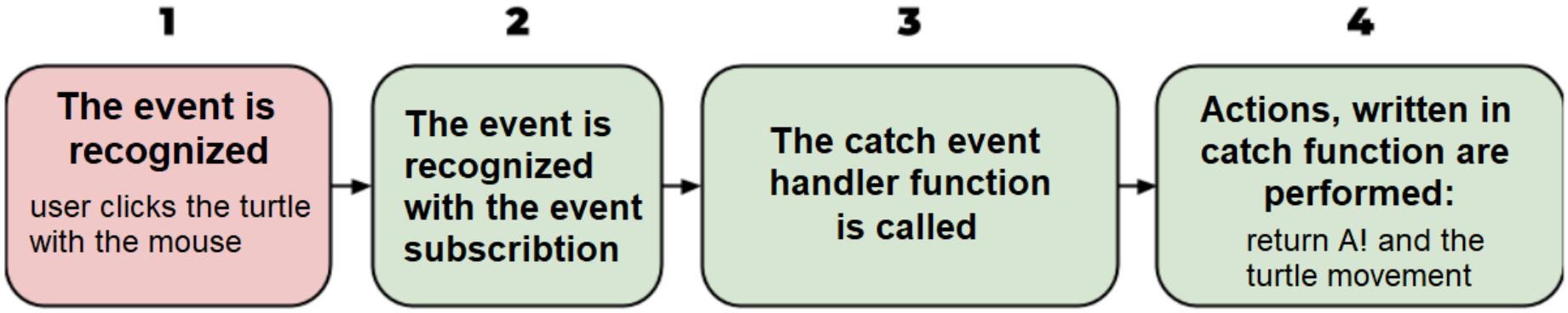
If an event occurs, the event subscription is triggered. The program execution system then switches the algorithm to a function that performs certain actions.

How to use:

For example, if we want the turtle to move around the screen when we click on it, we need to write a function that the program will reference when we click on it. After that, we need to create an event subscription.



Let's consider an example. In the program below, the turtle module and random were connected. An object t of type Turtle was created, the Turtle shape was set and the color was changed to red, and the feather was raised so that the executor would not leave a trace.




```
#a function the system refers to
#commands executed when the turtle is clicked
def catch(x, y):
    t.write('A!')
    new_x = randint(-100, 100)
    new_y = randint(-100, 100)
    t.goto(new_x, new_y)

#"turtle click" event subscription
t.onclick(catch)
```